CSCI 377: Computer Algorithms

John Jay College of Criminal Justice — CUNY

Project Title: Cash Flow Minimizer System

Team Members: Mahbubur Rahman

GitHub Repository: https://github.com/MAHBUB98R/CSCI377-Cash-Flow-Minimizer.git

Date Submitted: 8 May, 2026

Abstract

This project focuses on designing and implementing a Cash Flow Minimizer system that reduces the number of transactions required to settle debts among a group of people. In many real-world situations, people borrow money from each other during trips, shared expenses, or group activities, which can create complicated payment chains. The system calculates the net balance of each participant and generates a simplified set of transactions.

Two algorithms were implemented and compared in this project. The first algorithm uses a basic greedy approach that repeatedly scans all balances to find the maximum debtor and creditor. The second algorithm improves efficiency by using heap-based priority queues. The project mainly uses lists, dictionaries, and heaps as data structures.

Experimental testing was performed using different input sizes to compare the runtime of both algorithms. The results showed that the heap-based algorithm performed better for larger datasets and matched the expected theoretical time complexity. Overall, the project demonstrates how algorithm optimization and data structure selection can improve the efficiency of solving real-world financial problems.

1. Introduction

1.1 Problem Statement

When groups of people share expenses, many transactions happen between individuals. Over time, it becomes difficult to track who owes money and how much should be paid back. In many cases, people make unnecessary payments to multiple individuals even though the debts could be simplified.

The purpose of this project is to create a Cash Flow Minimizer system that reduces the total number of transactions needed to settle debts. The system calculates the net balance of each person and produces an optimized payment plan.

1.2 Project Objectives

- Implement a slow greedy cash flow minimization algorithm
- Implement a faster heap-based optimization algorithm
- Compare the runtime performance of both algorithms
- Reduce the number of transactions between users
- Analyze theoretical and practical time complexity results

1.3 Topic Selection Rationale

I selected the Cash Flow Minimizer topic because it focuses strongly on algorithms and optimization. The project also connects well to real world situations where people share expenses and need an easier way to settle debts. Another reason for choosing this topic is that it allows a clear comparison between a slower algorithm and a more optimized algorithm using different data structures.

2. System Architecture

2.1 Data Structures Used

Data Structure Data	**Role in System**	**Justification & Key Complexity**
List	*Stores all transactions*	*Easy to manage sequential transaction data*
Dictionary / Hash Map	*Stores net balances*	*Allows fast O(1) updates and lookups*
Heap / Priority Queue	*Stores creditors and debtors in fast algorithm*	*Allows efficient retrieval of maximum debtor and creditor in O(log n)*

2.2 Module / Component Description

Input Module

Handles transaction data entered into the system.

Balance Calculation Module

Calculates the net balance of each participant by processing all transactions.

Slow Greedy Algorithm Module

Uses repeated scanning to identify the maximum debtor and creditor.

Heap-Based Optimization Module

Uses heaps to improve the efficiency of selecting debtors and creditors.

Output Module

Displays minimized transactions and runtime results.

3. Algorithm Design

3.1 Algorithm Selection & Rationale

Algorithm 1- Slow Greedy Algorithm

The first algorithm uses a greedy approach to settle debts. It repeatedly scans through all balances to find the person with the highest debt and the person with the highest credit. The minimum possible amount is transferred between them until all balances become zero.

This algorithm was chosen because it is simple and easy to understand. However, repeated scanning makes it slower for large datasets.

Algorithm 2 - Heap Based Algorithm

The second algorithm improves the greedy approach by using heaps, also called priority queues. Instead of scanning all balances repeatedly, the algorithm efficiently removes the largest debtor and creditor from heaps.

This algorithm was selected because it improves runtime efficiency and performs better with larger input sizes.

3.2 Pseudocode

Algorithm 1: Slow Greedy Cash Flow Minimizer

Input: List of transactions
Output: Optimized transaction list

```
1. Calculate net balance for each person
2. WHILE balances are not zero:
3.     Find maximum creditor
4.     Find maximum debtor
5.     Transfer minimum amount
6.     Update balances
7.     Store transaction
8. END WHILE
9. RETURN optimized transactions
```

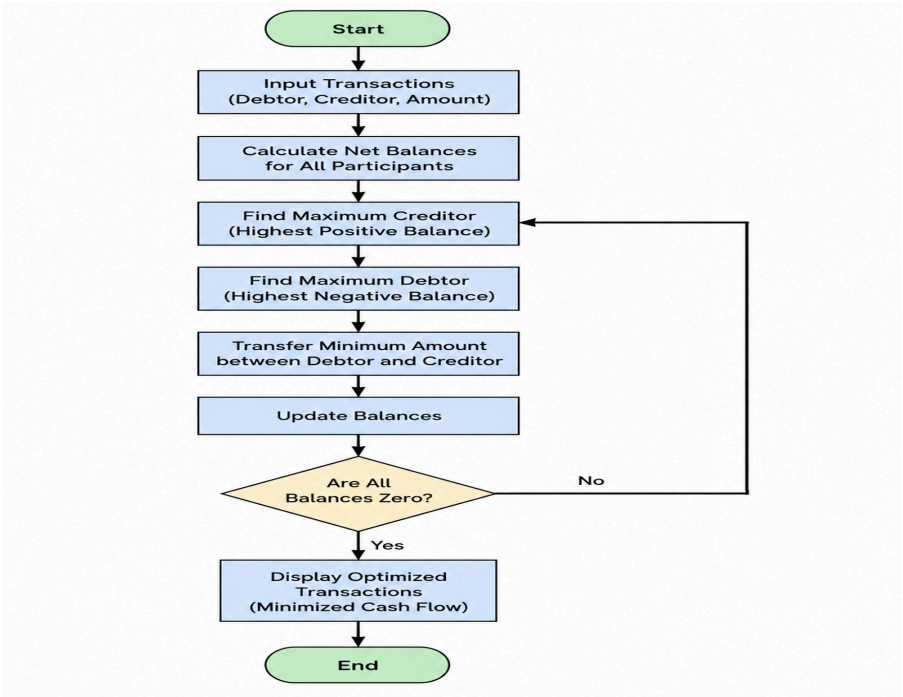
Algorithm 2: Heap-Based Cash Flow Minimizer

Input: List of transactions

Output: Optimized transaction list

```
1. Calculate net balances
2. Insert creditors into max heap
3. Insert debtors into min heap
4. WHILE heaps are not empty:
5.     Remove largest creditor
6.     Remove largest debtor
7.     Transfer minimum amount
8.     Update balances
9.     Reinsert remaining balances
10. END WHILE
11. RETURN optimized transactions
```

3.3 Flowchart / Process Diagram



The flowchart shows the overall process of the Cash Flow Minimizer system. The program starts by taking transaction input and calculating the net balance for each participant. The algorithm then repeatedly identifies the maximum creditor and maximum debtor, transfers the minimum possible amount between them, and updates balances until all balances become zero. Finally, the optimized transactions are displayed.

4. Time Complexity Analysis

Algorithm	Best Case	Average Case	Worst Case
Slow Greedy Algorithm	O(n)	$O(n^2)$	$O(n^2)$
Heap-Based Algorithm	O(n log n)	O(n log n)	O(n log n)

Slow Greedy Algorithm Analysis

The slow greedy algorithm repeatedly scans all balances to find the maximum debtor and maximum creditor. Each scan takes O(n) time, and the process may repeat up to n times. Because of this repeated scanning, the overall worst-case time complexity becomes $O(n^2)$.

Heap Based Algorithm Analysis

The heap-based algorithm uses priority queues to efficiently retrieve the maximum debtor and creditor. Heap insertion and removal operations take O(log n) time. Since multiple heap operations occur during processing, the total runtime becomes O(n log n), making it more efficient for larger datasets.

5. Experimental Setup and Results

5.1 Timing Methodology

The runtime of both algorithms was measured using Python's time.perf_counter() function. Only the execution time of the algorithms was measured, excluding printing and user input. Each experiment was repeated three times, and the average runtime was recorded in milliseconds. Randomly generated transaction datasets were used to test different input sizes.

5.2 Results Table

Record measured runtimes for each input size and algorithm.

Input Size	Slow Algorithm (ms)	Fast Algorithm (ms)
100	0.9361	0.126
500	19.7017	0.7088

1000	81.4272	1.3979
5000	2025.1676	7.8583
10000	8891.147	22.1174

5.3 Interpretation: Theory vs. Practice

The experimental results generally matched the expected theoretical time complexity. As the input size increased, the runtime of the slow greedy algorithm increased much faster compared to the heap based algorithm. This showed the impact of repeated scanning operations in the slower version.

The heap based algorithm consistently performed better for larger datasets because heap operations are more efficient than repeatedly searching through all balances. Although both algorithms produced the same final optimized transactions, the faster algorithm handled large inputs more effectively.

Some small variations in timing results may have been caused by system performance, memory usage, and Python interpreter overhead. Overall, the experiments confirmed that better data structures can significantly improve algorithm efficiency.

6. Design Decisions and Discussion

6.1 Challenges Encountered

One challenge during the project was correctly updating balances after each transaction. Small mistakes in balance updates caused incorrect outputs during testing. This issue was solved through debugging and testing with smaller datasets.

Another challenge was implementing the heap-based algorithm correctly. Managing heap

insertion and removal operations required careful handling of updated balances.

6.3 Limitations

- The system currently uses randomly generated transactions instead of real financial datasets.
- Very large datasets may still increase runtime and memory usage.
- The project focuses only on minimizing transactions and does not include a graphical user interface.

7. Reflection and Lessons Learned

7.1 What Worked Well

The overall structure of the project worked successfully, and both algorithms produced correct optimized transaction outputs. The heap based algorithm showed clear performance improvements for larger datasets. The experiments also helped strengthen understanding of time complexity and algorithm optimization.

7.2 What You Would Do Differently

If I were to improve this project further, I would add a graphical user interface and support file-based input. I would also test the system using larger real-world financial datasets to better evaluate practical performance.

7.3 Individual Reflection

Mahbubur Rahman

I completed all parts of this project independently, including algorithm design, coding, testing, and report writing. Through this project, I learned more about greedy algorithms, heaps, and time complexity analysis. Comparing two algorithms helped me better understand how data structures affect efficiency and performance. This project also improved my confidence in designing and analyzing algorithms for real-world problems.

8. Generative AI Usage Disclosure

Did you use Generative AI tools?

- Yes

Tool Used: ChatGPT

Purpose: Brainstorming project structure, understanding algorithms, improving report organization, and verifying pseudocode.

How Output Was Verified: All generated ideas, explanations, and code were reviewed, modified, tested, and verified manually before being included in the project.

9. GitHub Repository

9.1 Repository Link: https://github.com/MAHBUB98R/CSCI377-Cash-Flow-Minimizer.git